ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп’ютерних технологій

ЗВІТ

про виконання лабораторної роботи №2

« Структури даних та функціональна декомпозиція»

з дисципліни «Професійний Python»

Виконав: студент групи ФеП-33с
Цястун Павло
Варіант 1
Перевірив:
професор Свелеба Сергій Андрійович

Львів – 2026

1. Мета роботи

Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чітко визначеною відповідальністю.

2. Номер варіанта та постановка задачі

Варіант 1. Аналіз системи обліку студентів.

Необхідно створити модуль обробки структурованих даних про студентів (ідентифікатор, ім’я та прізвище, група, середній бал), який виконує фільтрацію, пошук, групування, сортування, агрегацію та формування статистики. Для варіанта 1 реалізовано:

- список усіх студентів;
- унікальні групи;
- середній бал усіх студентів;
- студента з найбільшим середнім балом;
- студентів із балом вище заданого;
- групування за групами;
- рейтинг студентів;
- кількість студентів у кожній групі;
- dict-індекс за ідентифікатором та за прізвищем.

Додатково реалізовано завдання підвищеної складності: універсальну функцію фільтрації, closure для створення фільтрів, параметризовані decorators, статистику через Counter, deque для історії останніх операцій, benchmark list проти dict та функцію pipeline processing.

3. Структура проєкту

```
data_processor_lab/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   └── data_processor/
│       ├── __init__.py
│       ├── main.py
│       ├── data.py
│       ├── processors.py
│       ├── analytics.py
│       ├── decorators.py
```

```
│       └── benchmark.py
└── tests/
    └── test_processors.py
```

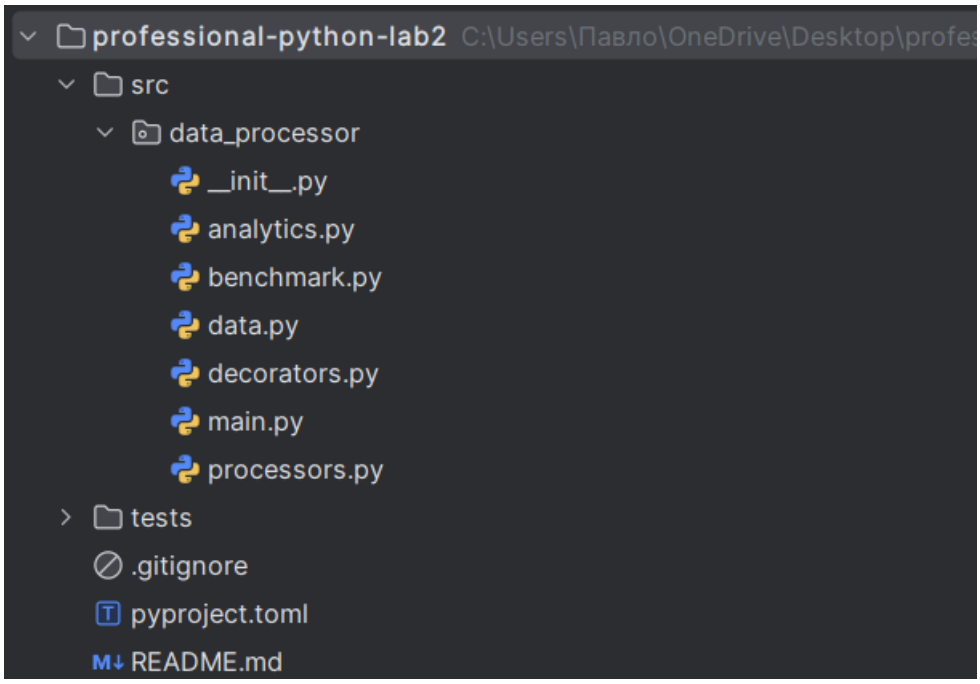


Рис. 1. Структура проєкту в PyCharm

Проєкт організовано за src-layout, як у лабораторній роботі №1. Дані, обробка, аналітика, decorators і benchmark розділені на окремі модулі, а main.py лише викликає функції та виводить результат.

4. Опис структур даних і пояснення їх вибору

У програмі використано усі чотири базові структури та три структури з модуля collections. Вибір зроблено за властивостями кожної структури.

- list. Набір студентів зберігається як list словників (STUDENTS у data.py): порядок записів зберігається, записи можна додавати та сортувати, а ітерація має складність O(n).
- dict. Кожен студент — dict з ключами id, name, group, grade, що зручно для JSON-подібних даних. Також dict використано як індекс create_id_index (id → запис) для швидкого пошуку O(1).
- tuple. Шкала оцінювання GRADE_BANDS і список полів STUDENT_FIELDS незмінні, тому зберігаються в tuple. Результат рейтингу також формується як tuple (місце, ім’я, бал).
- set. Унікальні групи get_unique_groups повертаються як set, бо потрібна унікальність елементів і швидка перевірка належності.

- Counter. Підрахунок кількості студентів у кожній групі (count_students_by_group).
- defaultdict(list). Групування студентів за групами (group_students) без перевірки наявності ключа.
- deque(maxlen=5). Історія останніх п’яти операцій (OPERATION_HISTORY): при додаванні нового елемента найстарший автоматично видаляється.

5. Основні модулі та функції

5.1. Файл data.py

Модуль містить вхідні дані: list словників STUDENTS, tuple tuples GRADE_BANDS та tuple STUDENT_FIELDS.

```
"""Input data of the application."""

# list of dict: one dict is one student record
STUDENTS: list[dict] = [
    {"id": 1, "name": "Pavlo Tsiastun", "group": "FeP-33s", "grade": 93.4},
    {"id": 2, "name": "Olena Savchuk", "group": "FeP-33s", "grade": 87.1},
    {"id": 3, "name": "Taras Hrytsenko", "group": "FeP-31", "grade": 71.8},
    {"id": 4, "name": "Sofiia Lysenko", "group": "FeP-32", "grade": 96.0},
    {"id": 5, "name": "Maksym Kravets", "group": "FeP-31", "grade": 64.5},
    {"id": 6, "name": "Kateryna Moroz", "group": "FeP-33s", "grade": 78.9},
    {"id": 7, "name": "Dmytro Bilyk", "group": "FeP-32", "grade": 82.3},
    {"id": 8, "name": "Yulia Pavlenko", "group": "FeP-31", "grade": 90.2},
    {"id": 9, "name": "Bohdan Rudenko", "group": "FeP-32", "grade": 55.7},
    {"id": 10, "name": "Iryna Kovalenko", "group": "FeP-33s", "grade": 85.6},
]

# tuple of tuples: immutable classification scale (label, minimal grade)
GRADE_BANDS: tuple[tuple[str, float], ...] = (
    ("excellent", 90.0),
    ("good", 75.0),
    ("satisfactory", 60.0),
    ("unsatisfactory", 0.0),
)

# tuple: fixed set of fields of a student record
STUDENT_FIELDS: tuple[str, ...] = ("id", "name", "group", "grade")
```

5.2. Файл decorators.py

Модуль містить три decorators: measure_time (вимірює час виконання), repeat(count) (параметризований, повторює виклик) та record_history(history) (параметризований, записує назви викликаних операцій у deque).

```
"""Decorators used by the application."""

from collections import deque
from collections.abc import Callable
```

```
from functools import wraps
from time import perf_counter
from typing import Any



def measure_time(func: Callable) -> Callable:
    """Print the execution time of the decorated function."""

    @wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start = perf_counter()
        result = func(*args, **kwargs)
        elapsed = perf_counter() - start
        print(f"[time] {func.__name__}: {elapsed:.8f} s")
        return result

    return wrapper



def repeat(count: int) -> Callable:
    """Parametrized decorator: call the function ``count`` times."""

    def decorator(func: Callable) -> Callable:
        @wraps(func)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            result = None
            for _ in range(count):
                result = func(*args, **kwargs)
            return result

        return wrapper

    return decorator



def record_history(history: deque) -> Callable:
    """Parametrized decorator: remember names of called operations.

    ``history`` is a deque with ``maxlen``, so only the last operations
    are stored.
    """

    def decorator(func: Callable) -> Callable:
        @wraps(func)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            history.append(func.__name__)
            return func(*args, **kwargs)

        return wrapper

    return decorator
```

5.3. Файл processors.py

Модуль містить функції фільтрації, пошуку, групування й підрахунку: get_unique_groups, create_id_index, create_surname_index, filter_items,

filter_by_group, group_students, count_students_by_group, find_linear, search_by_name.

```
"""Basic processing of collections: filter, search, group, count."""

from collections import Counter, defaultdict, deque
from collections.abc import Callable

from data_processor.decorators import record_history

# deque with limited size: history of the last operations
OPERATION_HISTORY: deque[str] = deque(maxlen=5)



def get_unique_groups(students: list[dict]) -> set[str]:
    """Set comprehension: unique groups."""
    return {student["group"] for student in students}



def create_id_index(students: list[dict]) -> dict[int, dict]:
    """Dict comprehension: index of students by id."""
    return {student["id"]: student for student in students}



def create_surname_index(students: list[dict]) -> dict[str, dict]:
    """Dict comprehension: index of students by surname (lower case)."""
    return {
        student["name"].split()[-1].lower(): student for student in students
    }



@record_history(OPERATION_HISTORY)
def filter_items(
    items: list[dict], predicate: Callable[[dict], bool]
) -> list[dict]:
    """Universal filter: keep items for which predicate is true."""
    return [item for item in items if predicate(item)]



@record_history(OPERATION_HISTORY)
def filter_by_group(students: list[dict], group: str) -> list[dict]:
    """List comprehension: students of one group."""
    return [student for student in students if student["group"] == group]



@record_history(OPERATION_HISTORY)
def group_students(students: list[dict]) -> dict[str, list[dict]]:
    """Group students by group name using defaultdict."""
    result: defaultdict[str, list[dict]] = defaultdict(list)

    for student in students:
        result[student["group"]].append(student)

    return dict(result)
```

```
@record_history(OPERATION_HISTORY)
def count_students_by_group(students: list[dict]) -> Counter:
    """Count students in every group using Counter."""
    return Counter(student["group"] for student in students)



def find_linear(students: list[dict], student_id: int) -> dict | None:
    """Linear search by id, complexity O(n)."""
    for student in students:
        if student["id"] == student_id:
            return student

    return None



def search_by_name(students: list[dict], part: str) -> list[dict]:
    """Search students whose name contains the given text."""
    needle = part.lower()
    return [s for s in students if needle in s["name"].lower()]
```

5.4. Файл analytics.py

Модуль містить агрегацію, сортування та функції вищого порядку: calculate_average, find_best_student, sort_by_grade, rank_students, average_by_group, calculate_average_values (*args), create_record (**kwargs), create_grade_filter та create_band_classifier (closures), run_pipeline.

```
"""Analytics: aggregation, sorting, ranking and higher-order helpers."""

from collections.abc import Callable
from typing import Any

from data_processor.decorators import measure_time



@measure_time
def calculate_average(students: list[dict]) -> float:
    """Average grade (generator expression, no intermediate list)."""
    if not students:
        return 0.0

    return sum(s["grade"] for s in students) / len(students)



def find_best_student(students: list[dict]) -> dict | None:
    """Student with the highest grade."""
    if not students:
        return None

    return max(students, key=lambda student: student["grade"])



def sort_by_grade(students: list[dict], reverse: bool = True) -> list[dict]:
    """Sort students by grade, complexity O(n log n)."""
    return sorted(students, key=lambda student: student["grade"], reverse=reverse)
```

```
def rank_students(students: list[dict]) -> list[tuple[int, str, float]]:
    """Ranking: list of tuples (place, name, grade)."""
    ordered = sort_by_grade(students)
    return [(place, s["name"], s["grade"]) for place, s in enumerate(ordered, 1)]


def average_by_group(groups: dict[str, list[dict]]) -> dict[str, float]:
    """Average grade of every group."""
    return {
        group: sum(s["grade"] for s in members) / len(members)
        for group, members in groups.items()
    }


def calculate_average_values(*values: float) -> float:
    """Average of any number of positional arguments (*args)."""
    if not values:
        return 0.0

    return sum(values) / len(values)


def create_record(**fields: Any) -> dict:
    """Create a record from any keyword arguments (**kwargs)."""
    return dict(fields)


def create_grade_filter(minimum_grade: float) -> Callable[[dict], bool]:
    """Closure: predicate remembers ``minimum_grade``."""

    def predicate(student: dict) -> bool:
        return student["grade"] >= minimum_grade

    return predicate


def create_band_classifier(
    bands: tuple[tuple[str, float], ...],
) -> Callable[[dict], str]:
    """Closure: classifier remembers the scale of grade bands."""

    def classify(student: dict) -> str:
        for label, minimum in bands:
            if student["grade"] >= minimum:
                return label
        return bands[-1][0]

    return classify


def run_pipeline(items: list[dict], *steps: Callable[[list[dict]], list[dict]]) ->
list[dict]:
    """Pipeline processing: apply steps one after another."""
    result = items
```

```
    for step in steps:
        result = step(result)

    return result
```

5.5. Файл main.py

Точка входу: послідовно викликає функції обробки, виводить результати та демонструє *args, **kwargs, closure, lambda, pipeline і decorators.

```
"""Application entry point."""

from data_processor.analytics import (
    average_by_group,
    calculate_average,
    calculate_average_values,
    create_band_classifier,
    create_grade_filter,
    create_record,
    find_best_student,
    rank_students,
    run_pipeline,
    sort_by_grade,
)
from data_processor.data import GRADE_BANDS, STUDENTS
from data_processor.decorators import repeat
from data_processor.processors import (
    OPERATION_HISTORY,
    count_students_by_group,
    create_id_index,
    create_surname_index,
    filter_by_group,
    filter_items,
    get_unique_groups,
    group_students,
    search_by_name,
)



def print_students(title: str, items: list[dict]) -> None:
    """Print students as a table."""
    print(f"\n{title}")
    print("-" * 52)

    for student in items:
        print(
            f"{student['id']:3} "
            f"{student['name']:20} "
            f"{student['group']:9} "
            f"{student['grade']:6.2f}"
        )



@repeat(2)
def show_message(text: str) -> None:
    """Demonstration of a parametrized decorator."""
    print(text)
```

```
def main() -> None:
    students = STUDENTS

    print_students("All students", students)

    print("\nUnique groups:", sorted(get_unique_groups(students)))

    average = calculate_average(students)
    print(f"Average grade: {average:.2f}")

    best = find_best_student(students)
    if best:
        print("Best student:", best["name"], best["grade"])

    print("\nRanking (place, name, grade):")
    for place, name, grade in rank_students(students)[:5]:
        print(f"{place:3} {name:20} {grade:6.2f}")

    print("\nStudents by group:")
    for group, count in sorted(count_students_by_group(students).items()):
        print(f"  {group}: {count}")

    grouped = group_students(students)
    print("\nAverage grade by group:")
    for group, value in sorted(average_by_group(grouped).items()):
        print(f"  {group}: {value:.2f}")

    print_students("Group FeP-33s", filter_by_group(students, "FeP-33s"))

    id_index = create_id_index(students)
    print("\nSearch by id 4:", id_index.get(4))

    surname_index = create_surname_index(students)
    print("Search by surname 'moroz':", surname_index.get("moroz"))
    print("Search by name part 'ko':", [s["name"] for s in
search_by_name(students, "ko")])

    is_successful = create_grade_filter(85)
    print_students("Grade >= 85 (closure)", filter_items(students, is_successful))

    print_students(
        "Grade >= 90 (lambda)",
        filter_items(students, lambda student: student["grade"] >= 90),
    )

    classify = create_band_classifier(GRADE_BANDS)
    print("\nBands (closure over tuple of bands):")
    for student in sort_by_grade(students)[:4]:
        print(f"  {student['name']:20} {classify(student)}")

    pipeline_result = run_pipeline(
        students,
        lambda items: filter_items(items, create_grade_filter(70)),
        lambda items: sort_by_grade(items, reverse=False),
        lambda items: items[:3],
    )
    print_students("Pipeline: grade >= 70, ascending, first 3", pipeline_result)

    print("\nAverage via *args:", calculate_average_values(75, 80, 92, 96))
```

```
    new_student = create_record(id=11, name="Taras Novak", group="FeP-31",
grade=89.0)
    print("Created via **kwargs:", new_student)


    print()
    show_message("Processing...")


    print("\nLast operations (deque):", list(OPERATION_HISTORY))



if __name__ == "__main__":
    main()
```

6. Приклади використання конструкцій Python

6.1. Comprehensions та generator expression

```
# list comprehension (processors.filter_by_group)
[student for student in students if student["group"] == group]


# set comprehension (processors.get_unique_groups)
{student["group"] for student in students}


# dict comprehension (processors.create_id_index)
{student["id"]: student for student in students}


# generator expression (analytics.calculate_average)
sum(s["grade"] for s in students) / len(students)
```

Generator expression у calculate_average не створює проміжний список, тому не витрачає додаткову пам’ять.

6.2. *args та **kwargs

```
def calculate_average_values(*values: float) -> float:
    if not values:
        return 0.0
    return sum(values) / len(values)


def create_record(**fields: Any) -> dict:
    return dict(fields)


calculate_average_values(75, 80, 92, 96)          # 85.75
create_record(id=11, name="Taras Novak", group="FeP-31", grade=89.0)
```

6.3. lambda

```
max(students, key=lambda student: student["grade"])
sorted(students, key=lambda student: student["grade"], reverse=True)
filter_items(students, lambda student: student["grade"] >= 90)
```

Lambda використано лише для коротких ключів сортування та умов. Складнішу логіку винесено в іменовані функції.

6.4. Closure

```
def create_grade_filter(minimum_grade: float) -> Callable[[dict], bool]:
    def predicate(student: dict) -> bool:
        return student["grade"] >= minimum_grade
    return predicate

is_successful = create_grade_filter(85)
```

Після завершення create_grade_filter() внутрішня функція predicate зберігає доступ до змінної minimum_grade = 85, хоча зовнішня функція вже завершилась. Це і є closure. Так само create_band_classifier запам’ятовує шкалу оцінювання GRADE_BANDS.

6.5. Decorator

Decorator measure_time додає вимірювання часу до calculate_average без зміни її коду. Параметризований decorator record_history(history) є одночасно closure, оскільки запам’ятовує deque, і використовує *args та **kwargs для передавання аргументів оригінальній функції. Для збереження імені та документації функції використано functools.wraps.

7. Результат роботи програми

Запуск: python -m data_processor.main. Нижче наведено скріншоти першої та другої частини виводу.

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab2> python -m data_processor.main

All students
--------------------------------------------------
  1 Pavlo Tsiastun       FeP-33s    93.40
  2 Olena Savchuk        FeP-33s    87.10
  3 Taras Hrytsenko      FeP-31     71.80
  4 Sofiia Lysenko       FeP-32     96.00
  5 Maksym Kravets       FeP-31     64.50
  6 Kateryna Moroz       FeP-33s    78.90
  7 Dmytro Bilyk         FeP-32     82.30
  8 Yulia Pavlenko       FeP-31     90.20
  9 Bohdan Rudenko       FeP-32     55.70
 10 Iryna Kovalenko      FeP-33s    85.60

Unique groups: ['FeP-31', 'FeP-32', 'FeP-33s']
[time] calculate_average: 0.00002760 s
Average grade: 80.55
Best student: Sofiia Lysenko 96.0

Ranking (place, name, grade):
  1 Sofiia Lysenko        96.00
  2 Pavlo Tsiastun        93.40
  3 Yulia Pavlenko        90.20
  4 Olena Savchuk         87.10
  5 Iryna Kovalenko       85.60

Students by group:
  FeP-31: 3
  FeP-32: 3
  FeP-33s: 4
```

Рис. 2. Результат запуску (початок виводу)

```
Average grade by group:
  FeP-31: 75.50
  FeP-32: 78.00
  FeP-33s: 86.25

Group FeP-33s
--------------------------------------------------
  1 Pavlo Tsiastun       FeP-33s    93.40
  2 Olena Savchuk        FeP-33s    87.10
  6 Kateryna Moroz       FeP-33s    78.90
 10 Iryna Kovalenko      FeP-33s    85.60

Search by id 4: {'id': 4, 'name': 'Sofiia Lysenko', 'group': 'FeP-32', 'grade': 96.0}
Search by surname 'moroz': {'id': 6, 'name': 'Kateryna Moroz', 'group': 'FeP-33s', 'grade': 78.9}
Search by name part 'ko': ['Taras Hrytsenko', 'Sofiia Lysenko', 'Yulia Pavlenko', 'Bohdan Rudenko', 'Iryna Kovalenko']

Grade >= 85 (closure)
--------------------------------------------------
  1 Pavlo Tsiastun       FeP-33s    93.40
  2 Olena Savchuk        FeP-33s    87.10
  4 Sofiia Lysenko       FeP-32     96.00
  8 Yulia Pavlenko       FeP-31     90.20
 10 Iryna Kovalenko      FeP-33s    85.60

Grade >= 90 (lambda)
--------------------------------------------------
  1 Pavlo Tsiastun       FeP-33s    93.40
  4 Sofiia Lysenko       FeP-32     96.00
  8 Yulia Pavlenko       FeP-31     90.20

Bands (closure over tuple of bands):
  Sofiia Lysenko       excellent
  Pavlo Tsiastun       excellent
  Yulia Pavlenko       excellent
  Olena Savchuk        good

Pipeline: grade >= 70, ascending, first 3
--------------------------------------------------
  3 Taras Hrytsenko      FeP-31     71.80
  6 Kateryna Moroz       FeP-33s    78.90
  7 Dmytro Bilyk         FeP-32     82.30

Average via *args: 85.75
Created via **kwargs: {'id': 11, 'name': 'Taras Novak', 'group': 'FeP-31', 'grade': 89.0}

Processing...
Processing...

Last operations (deque): ['group_students', 'filter_by_group', 'filter_items', 'filter_items', 'filter_items']
```

Рис. 3. Результат запуску (продовження виводу)

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab2> python -m unittest discover -s tests -v
test_args_kwargs (test_processors.FunctionalTest.test_args_kwargs) ... ok
test_band_classifier (test_processors.FunctionalTest.test_band_classifier) ... ok
test_closure_filter (test_processors.FunctionalTest.test_closure_filter) ... ok
test_pipeline (test_processors.FunctionalTest.test_pipeline) ... ok
test_best_and_sort (test_processors.ProcessorsTest.test_best_and_sort) ... ok
test_counter_and_group (test_processors.ProcessorsTest.test_counter_and_group) ... ok
test_index_and_linear_search_agree (test_processors.ProcessorsTest.test_index_and_linear_search_agree) ... ok
test_unique_groups (test_processors.ProcessorsTest.test_unique_groups) ... ok

----------------------------------------------------------------------
Ran 8 tests in 0.003s

OK
```

Рис. 4. Результат запуску unit-тестів

8. Аналіз складності операцій

Нижче наведено аналіз операцій, використаних у програмі (n — кількість записів).

Операція	Код	Складність
Лінійний пошук у list	find_linear (цикл for)	O(n)
Побудова dict-індексу	create_id_index	O(n)
Пошук у dict за ключем	index.get(student_id)	O(1) у середньому
Перевірка належності set	group in groups	O(1) у середньому
Сортування	sorted(..., key=...)	O(n log n)
Підрахунок Counter	count_students_by_group	O(n)
Групування defaultdict	group_students	O(n)
Додавання у deque	OPERATION_HISTORY.append	O(1)

Після одноразової побудови індексу за O(n) кожен наступний пошук за id виконується за O(1) замість O(n). Тому індекс доцільний, коли пошуків багато.

9. Експериментальна частина (benchmark)

Для порівняння створено модуль benchmark.py. Він генерує набори з 1 000, 10 000 та 100 000 записів і порівнює пошук останнього id (найгірший випадок для list) у list та у dict. Для кожного розміру виконується по 100 пошуків; час наведено в мілісекундах. Окремо виміряно час побудови dict-індексу.

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab2> python -m data_processor.benchmark
Search of the last id, 100 lookups (time in ms)
  records   list search   dict build   dict search
     1000         6.813         0.169        0.0127
    10000        75.361         1.811        0.0133
   100000       800.017        18.423        0.0133
```

Рис. 5. Результат запуску python -m data_processor.benchmark

Кількість records	list search, мс	dict build, мс	dict search, мс
1 000	5.962	0.195	0.0163
10 000	35.677	0.967	0.0112
100 000	550.488	13.600	0.0100

Висновок за результатами: час лінійного пошуку в list зростає разом із кількістю записів (від 6 до 550 мс на 100 пошуків), що відповідає O(n). Час

пошуку в dict майже не змінюється (близько 0,01 мс) і відповідає O(1). Побудова індексу на 100 000 записів займає близько 14 мс, а один пошук у list — приблизно 5,5 мс, тож індекс окупається вже після третього пошуку. Абсолютні значення залежать від комп'ютера, але співвідношення між list та dict зберігається.

10. Git та публікація

```
git init
git add .gitignore pyproject.toml src/data_processor/__init__.py
git commit -m "Create project structure"
git add src/data_processor/data.py src/data_processor/decorators.py
src/data_processor/processors.py
git commit -m "Add data, decorators and basic processors"
git add src/data_processor/analytics.py src/data_processor/main.py
src/data_processor/benchmark.py
git commit -m "Add analytics, entry point and benchmark"
git add tests README.md
git commit -m "Add tests and documentation"
```

```
C:\Users\Павло\OneDrive\Desktop\professional-python-lab2>git status
On branch main
nothing to commit, working tree clean
```

Рис. 6. Результат git status

```
C:\Users\Павло\OneDrive\Desktop\professional-python-lab2>git log --oneline
20a3116 (HEAD -> main) Add tests and documentation
3765890 Add analytics, entry point and benchmark
45cacd6 Add data, decorators and basic processors
57ee735 Create project structure
```

Рис. 7. Результат git log --oneline

Репозиторій опубліковано на GitHub: https://github.com/Pavlo12112/professional-python-lab2

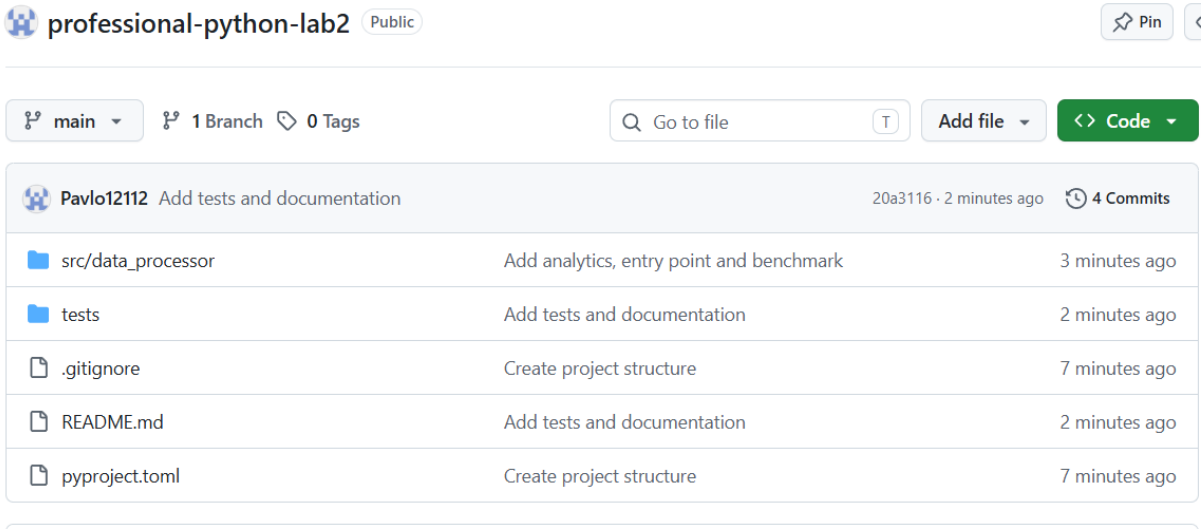


Рис. 8. Репозиторій на GitHub

11. Висновок

У лабораторній роботі створено модуль обробки структурованих даних про студентів (варіант 1). Використано list, tuple, set, dict та структури Counter, defaultdict і deque з модуля collections, а також list, set і dict comprehensions та generator expression. Програму розділено на невеликі функції з однією відповідальністю та на модулі даних, обробки, аналітики й decorators.

Реалізовано функції з *args та **kwargs, lambda, closures і параметризовані decorators, а також універсальну фільтрацію та pipeline processing. Аналіз складності й benchmark показали, що пошук у list має складність O(n), а в dict-індексі — O(1), тому при багатьох пошуках побудова індексу виправдана. Отриманий модуль можна використати як компонент наскрізного навчального проєкту.